

COMP 532

Machine Learning and Bioinspired
Optimisation
Lecture 10 Reinforcement Learning

Meng Fang

University of Liverpool



Overview

- Problem Solving from Nature
- Reinforcement Learning (RL)
 - Roots of RL
 - Key features of RL
 - Elements of RL
 - Multi-armed bandits
 - Solve the problem in Markov Decision Processes (MDPs)
 - Value function
 - Solve the Bellman Optimality Equation
 - Temporal Difference (TD) Learning
 - Others
- Deep Learning
- Multi-Agent Learning
- Swarm Intelligence

Recap: Value-Based RL (Q-learning)

- Core idea:
 - Learn a value function, then derive the policy.
- Update rule:
- $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$
- Key properties:
 - Bootstrapping
 - Off-policy
 - Fast learning
 - Biased but low variance
- Policy:
- $\pi(s) = \arg \max_a Q(s, a)$

Reward Shaping

- In many environments:
 - Reward only appears at the goal
 - Learning becomes slow
 - Exploration is inefficient
- Question:
 - Can we guide the agent without changing the optimal policy?

Reward Shaping

- Add additional reward:

$$r' = r + F(s, s')$$

- Reward designing:

- Provide heuristic knowledge by an additional reward

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + F(s, s') + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

- Potential-based reward shaping: [Additional rewards](#)

$$F(s, s') = \gamma(\Phi(s') - \Phi(s))$$

$F(s, s')$: Additional reward when moving from state s to s'

γ : Discount factor

$\Phi(s)$: Potential of state s

From 1-Step TD to Multi-Step Learning

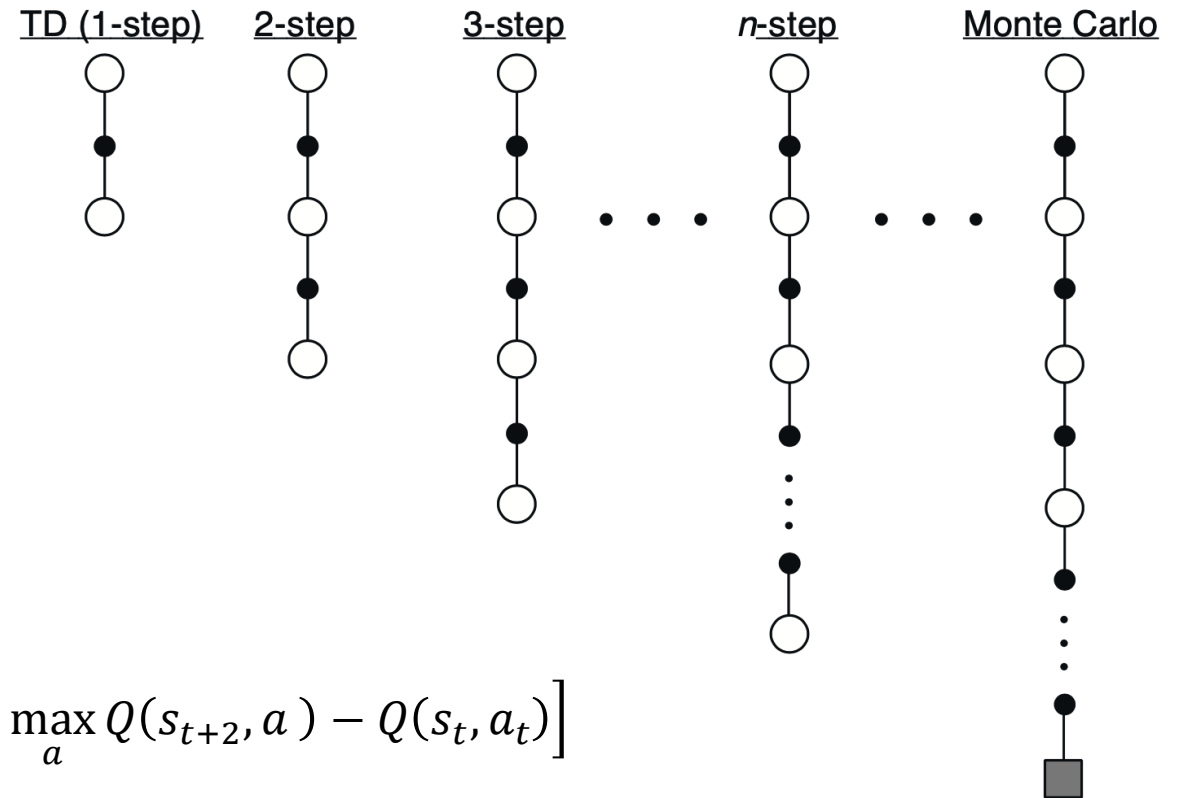
- TD(0):
 - Uses 1-step return
- Monte Carlo:
 - Uses full episode return
- General n-step return:
 - $G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n V(s_{t+n})$
- Trade-off:
 - Few steps $\rightarrow$ low variance, high bias
 - Many steps $\rightarrow$ low bias, high variance

Bias–Variance Tradeoff in RL

- TD(0):
 - High bias
 - Low variance
- Monte Carlo:
 - Low bias
 - High variance
- n-step methods:
 - Between the two
- This tradeoff is central in RL design.

Multi-Step TD

- So far we have looked at TD methods that backup values after each single time step.
- In general, we could backup values over a longer range of steps.



$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[\underbrace{r_{t+1} + \gamma r_{t+2} + \gamma^2 \max_a Q(s_{t+2}, a)}_{\text{2 steps}} - Q(s_t, a_t) \right]$$

2 steps

A Different Approach: Direct Policy Optimisation

- So far:
 - Learn value $\rightarrow$ derive policy
- Alternative:
 - Directly optimise the policy.
- Define objective:
 - $J(\theta) = \mathbb{E}_{\pi_{\theta}}[\sum \gamma^t r_t]$
- Policy gradient theorem:
 - $\nabla J(\theta) = \mathbb{E}[G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$
- This is REINFORCE.

Policy Gradient methods - Learning Without Value Functions

- Common among all methods we have discussed:
 - DP, MC, TD (Sarsa, Q-learning)
 - They all use value functions!
- Policy π is based on value functions
- We can also learn by directly manipulating the policy instead of first looking for the optimal value function.

Why Policy Gradient is Noisy

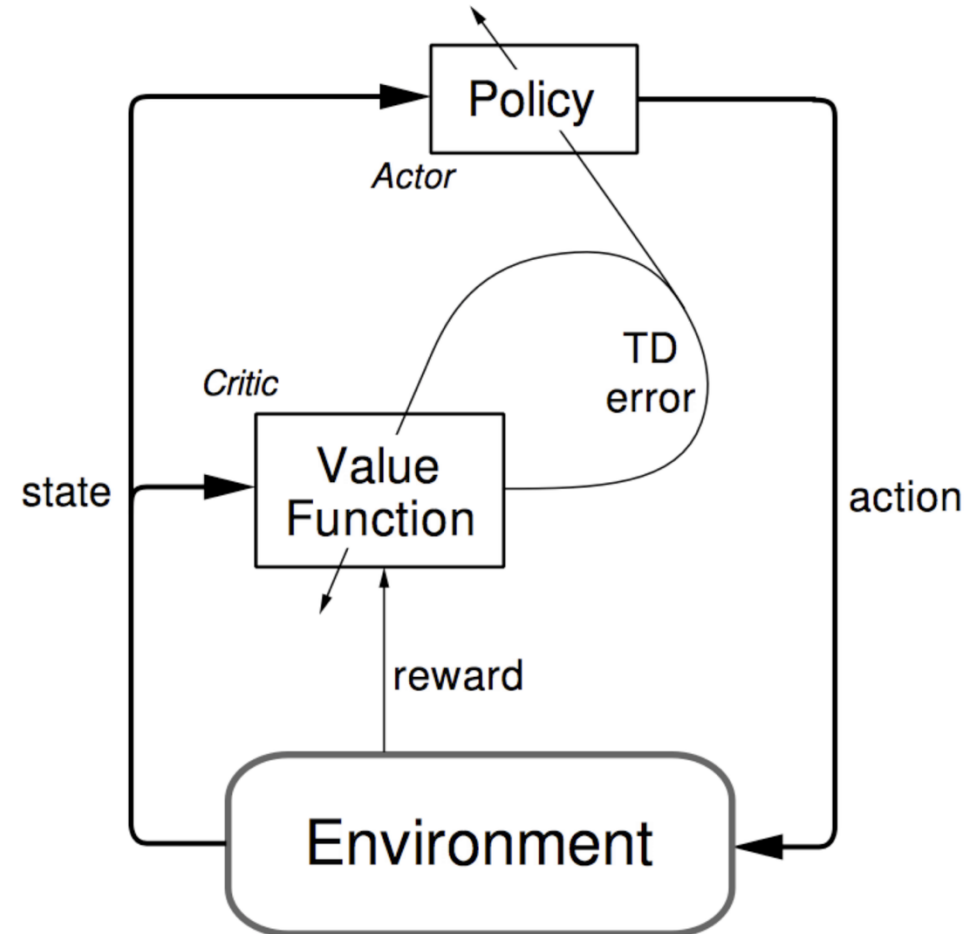
- Problem:
 - G_t is random
 - High variance updates
- Solution:
 - Subtract a baseline:

$$A_t = G_t - V(s_t)$$

- Updated gradient:
 - $\nabla J(\theta) = \mathbb{E}[A_t \nabla_{\theta} \log \pi(a_t | s_t)]$
- This reduces variance without changing expectation.

Actor–Critic: Best of Both Worlds

- Combine:
- Actor:
 - Updates policy
- Critic:
 - Estimates value function
- Actor update:
 - $\nabla_{\theta} \log \pi(a | s) A_t$
- Critic update:
 - $(G_t - V(s))^2$
- Advantages:
 - Lower variance than REINFORCE
 - Works in continuous action spaces
 - Foundation of modern RL



Actor–Critic Methods

- Actor

$$L_{actor} = - \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t) \underbrace{[G(s_t, a_t) - V_{\theta}^{\pi}(s_t)]}_{\text{Advantage}}$$

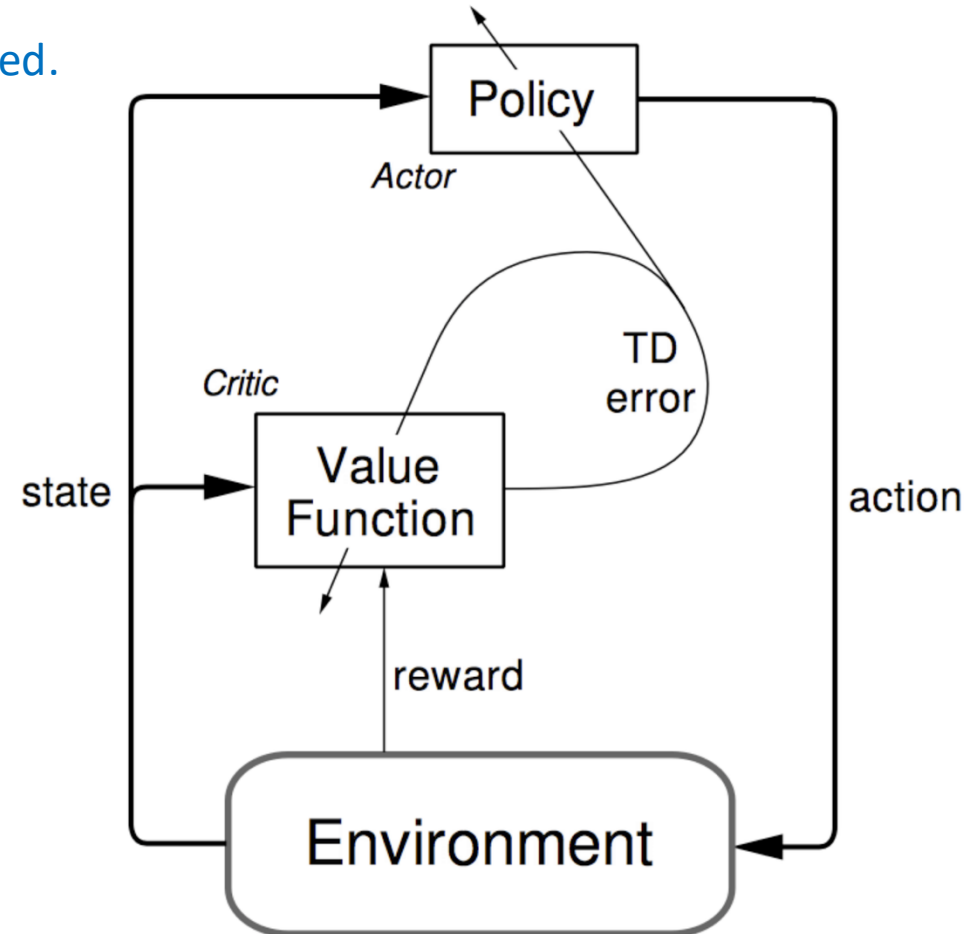
Now, the value function is parameterized.

- Critic

$$L_{critic} = L_{\delta}(G, V_{\theta}^{\pi})$$

where

$$G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} \quad \gamma \in (0,1)$$



Function Approximation

- Tabular methods fail when:
 - State space is large
 - State space is continuous
- Solution:
- Approximate value or policy with:
 - Neural networks
 - Gaussian processes
- But beware:
- The Deadly Triad
 - Off-policy learning
 - Bootstrapping
 - Function approximation
- Together they may cause instability.

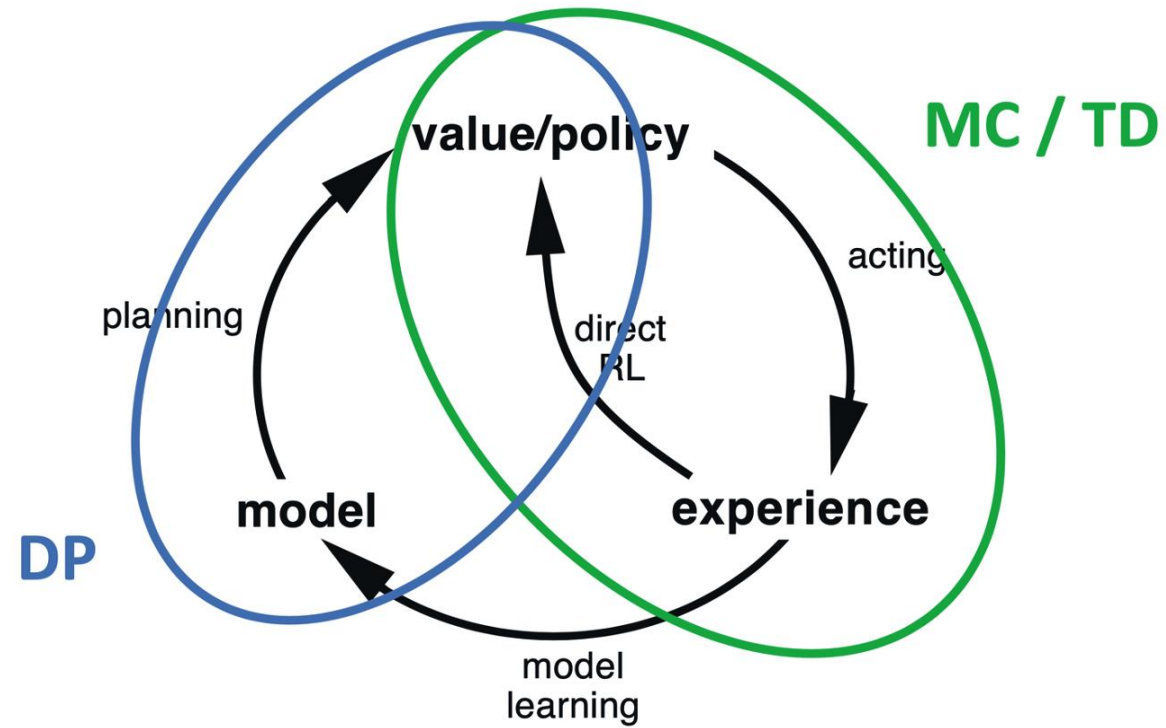
Model-Based RL

- So far: Learn only from experience
- If we have a model:
 - $P(s' | sa), R(s' a)$
- We can:
 - Plan
 - Simulate
 - Improve value without acting
- Examples: Dynamic Programming and Dyna-Q

Model-based methods trade:

- More computation
- Less environment interaction

Learning and Planning



Wrap-up

- Value-Based:
 - Q-learning
 - TD
 - n-step
 - Eligibility traces
- Policy-Based:
 - REINFORCE
 - Actor–Critic
- Scaling:
 - Function approximation